```
<!-- ... existing code ... -->
    <!-- Marked.js for Markdown parsing -->
    <script src="https://cdn.jsdelivr.net/npm/marked/marked.min.js"></script>
    <!-- Highlight.js for code syntax highlighting -->
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/highlight.js/11.7.0/styles/atom-one-dark.min.css">
    <script src="https://cdnjs.cloudflare.com/ajax/libs/highlight.js/11.7.0/highlight.min.js"></script>
    <!-- Google Fonts: Inter -->
    <link href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700&display=swap" rel="stylesheet">

    <!-- PWA Web App Manifest Link -->
    <link rel="manifest" href="manifest.json">
    <meta name="theme-color" content="#7c3aed">

    <script>
        tailwind.config = {
<!-- ... existing code ... -->
            <!-- User & Settings Footer -->
            <div class="p-3 border-t border-gray-800/80 flex items-center justify-between bg-darkcard/50">
                <div class="flex items-center space-x-3">
                    <div class="w-8 h-8 rounded-full bg-indigo-600 flex items-center justify-center text-xs font-semibold text-white">
                        FIDDY
                    </div>
                    <div class="truncate max-w-[110px]">
                        <p class="text-xs font-medium text-white truncate">Creator Mode</p>
                        <p id="mode-badge" class="text-[10px] text-gray-400">Mock Engine</p>
                    </div>
                </div>
                <button id="open-settings-btn" class="p-2 text-gray-400 hover:text-white hover:bg-gray-800 rounded-lg transition" title="Settings & Prompt Customization">
                    <i class="fa-solid fa-gear"></i>
                </button>
            </div>
        </aside>
<!-- ... existing code ... -->
```

```
    <!-- Settings & Custom System Prompt Modal -->
    <div id="settings-modal" class="fixed inset-0 bg-black/70 backdrop-blur-sm z-50 flex items-center justify-center p-4 hidden">
        <div class="bg-darkcard border border-gray-800 rounded-2xl max-w-lg w-full p-6 shadow-2xl relative max-h-[90vh] overflow-y-auto">
            <button id="close-settings-btn" class="absolute top-4 right-4 text-gray-400 hover:text-white">
                <i class="fa-solid fa-xmark text-lg"></i>
            </button>

            <div class="flex items-center space-x-3 mb-6">
                <div class="w-10 h-10 rounded-xl bg-indigo-600/20 border border-indigo-500/30 flex items-center justify-center text-indigo-400">
                    <i class="fa-solid fa-sliders text-lg"></i>
                </div>
                <div>
                    <h3 class="text-lg font-bold text-white">AI Configuration & Rules</h3>
                    <p class="text-xs text-gray-400">Customize Chart Fiddy Ac's brain & engine</p>
                </div>
            </div>

            <div class="space-y-4">
                <!-- Creator Identity & System Prompt Customizer -->
                <div>
                    <div class="flex justify-between items-center mb-1">
                        <label class="block text-xs font-semibold text-gray-300 uppercase">System Prompt (AI Brain)</label>
                        <button id="reset-prompt-btn" class="text-[10px] text-indigo-400 hover:underline">Reset to Default</button>
                    </div>
                    <textarea id="system-prompt-input" rows="6" class="w-full bg-darkbg border border-gray-700 rounded-xl p-3 text-xs text-gray-200 focus:outline-none focus:border-indigo-500 font-mono leading-relaxed resize-y"></textarea>
                    <p class="text-[11px] text-gray-400 mt-1">This prompt dictates how Chart Fiddy Ac behaves and ensures it credits <strong>Fiddy</strong> as its creator.</p>
                </div>

                <!-- API Provider Selection -->
```

```
            <div>
                <label class="block text-xs font-semibold text-gray-300 uppercase mb-1">Engine Mode</label>
                <select id="api-provider" class="w-full bg-darkbg border border-gray-700 rounded-xl px-3 py-2 text-sm text-white focus:outline-none focus:border-indigo-500">
                    <option value="mock">Built-in Intelligent Engine (Free / Offline Test)</option>
                    <option value="gemini">Google Gemini API (Live AI Responses)</option>
                </select>
            </div>

            <!-- API Key Input -->
            <div id="api-key-container" class="hidden">
                <label class="block text-xs font-semibold text-gray-300 uppercase mb-1">Google Gemini API Key</label>
                <input type="password" id="api-key-input" placeholder="AIzaSy..." class="w-full bg-darkbg border border-gray-700 rounded-xl px-3 py-2 text-sm text-white focus:outline-none focus:border-indigo-500">
            </div>

            <!-- Model Selection -->
            <div id="model-select-container" class="hidden">
                <label class="block text-xs font-semibold text-gray-300 uppercase mb-1">Model Version</label>
                <select id="model-name" class="w-full bg-darkbg border border-gray-700 rounded-xl px-3 py-2 text-sm text-white focus:outline-none focus:border-indigo-500">
                    <option value="gemini-1.5-flash">gemini-1.5-flash (Recommended)</option>
                    <option value="gemini-1.5-pro">gemini-1.5-pro (Advanced Reasoning)</option>
                </select>
            </div>
        </div>

        <div class="mt-6 flex justify-end gap-3">
            <button id="save-settings-btn" class="w-full bg-indigo-600 hover:bg-indigo-500 text-white font-medium py-2.5 rounded-xl text-sm transition shadow-lg shadow-indigo-600/30">
                Save System Prompt & Config
            </button>
        </div>
    </div>
```

```
    </div>

    <!-- JavaScript Application Logic -->
    <script>
        // Default Creator System Prompt explicitly identifying Fiddy as Creator
        const DEFAULT_SYSTEM_PROMPT = `
You are Chart Fiddy Ac—a warm, deeply supportive, engaging, highly versatile, and structured AI assistant created and designed by Fiddy.

Core Rules & Identity:
1. CREATOR IDENTITY: If anyone asks who created you, who built you, who designed you, or who your creator is, you MUST proudly state that you were created, designed, and developed by FIDDY.
2. PERSONALITY & STYLE: You combine Jimmy's warm, supportive, and encouraging tone with structured responses using clear markdown, headers, bullet points, tables, and code snippets.
3. VERSATILITY: You excel across academics (sciences, math, accounting), creative arts (acting monologues, songwriting guidance, storytelling), digital design, branding, and daily planning.
4. CREATIVE PROTECTION: NEVER auto-generate original creative works like full lyrics or scripts unless explicitly asked. Always protect and honor Fiddy and user creative voice.
        `.trim();

        // Application State
        let state = {
            systemPrompt: localStorage.getItem('cfa_system_prompt') ||
DEFAULT_SYSTEM_PROMPT,
            provider: localStorage.getItem('cfa_provider') || 'mock',
            apiKey: localStorage.getItem('cfa_api_key') || '',
            model: localStorage.getItem('cfa_model') || 'gemini-1.5-flash',
            chats: JSON.parse(localStorage.getItem('cfa_chats') || '[]'),
            currentChatId: null,
            isGenerating: false
        };

        // DOM Elements
        const messagesContainer = document.getElementById('messages-container');
        const welcomeScreen = document.getElementById('welcome-screen');
        const chatForm = document.getElementById('chat-form');
        const chatInput = document.getElementById('chat-input');
```

```
const sendBtn = document.getElementById('send-btn');
const charCount = document.getElementById('char-count');
const clearChatBtn = document.getElementById('clear-chat-btn');
const newChatBtn = document.getElementById('new-chat-btn');
const historyItems = document.getElementById('history-items');
const modeBadge = document.getElementById('mode-badge');

// Settings Elements
const settingsModal = document.getElementById('settings-modal');
const openSettingsBtn = document.getElementById('open-settings-btn');
const closeSettingsBtn = document.getElementById('close-settings-btn');
const saveSettingsBtn = document.getElementById('save-settings-btn');
const resetPromptBtn = document.getElementById('reset-prompt-btn');
const systemPromptInput = document.getElementById('system-prompt-input');
const apiProviderSelect = document.getElementById('api-provider');
const apiKeyInput = document.getElementById('api-key-input');
const apiKeyContainer = document.getElementById('api-key-container');
const modelSelectContainer = document.getElementById('model-select-container');
const modelNameSelect = document.getElementById('model-name');

// Register Service Worker for PWA
if ('serviceWorker' in navigator) {
    window.addEventListener('load', () => {
        navigator.serviceWorker.register('sw.js')
            .then(reg => console.log('PWA Service Worker registered'))
            .catch(err => console.log('Service Worker registration skipped'));
    });
}

// Initialize App
function initApp() {
    setupEventListeners();
    updateSettingsUI();

    if (state.chats.length === 0) {
        createNewChat();
    } else {
        state.currentChatId = state.chats[0].id;
```

```
        renderChatHistory();
        loadCurrentChat();
    }

    marked.setOptions({
        highlight: function(code, lang) {
            if (lang && hljs.getLanguage(lang)) {
                return hljs.highlight(code, { language: lang }).value;
            }
            return hljs.highlightAuto(code).value;
        },
        breaks: true
    });
}

function setupEventListeners() {
    chatInput.addEventListener('input', () => {
        chatInput.style.height = 'auto';
        chatInput.style.height = Math.min(chatInput.scrollHeight, 144) + 'px';
        charCount.textContent = `${chatInput.value.length} chars`;
    });

    chatInput.addEventListener('keydown', (e) => {
        if (e.key === 'Enter' && !e.shiftKey) {
            e.preventDefault();
            if (chatInput.value.trim() && !state.isGenerating) {
                chatForm.dispatchEvent(new Event('submit'));
            }
        }
    });

    chatForm.addEventListener('submit', (e) => {
        e.preventDefault();
        const text = chatInput.value.trim();
        if (!text || state.isGenerating) return;

        handleUserSendMessage(text);
        chatInput.value = '';
```

```
        chatInput.style.height = 'auto';
        charCount.textContent = '0 chars';
    });

    document.querySelectorAll('.prompt-chip').forEach(chip => {
        chip.addEventListener('click', () => {
            const text = chip.querySelector('span:last-child').textContent.replace(/^"|"$/g, '');
            chatInput.value = text;
            chatInput.focus();
            chatInput.dispatchEvent(new Event('input'));
        });
    });

    // Navigation
    document.getElementById('open-sidebar-btn').addEventListener('click', toggleSidebar);
    document.getElementById('close-sidebar-btn').addEventListener('click', toggleSidebar);
    document.getElementById('mobile-overlay').addEventListener('click', toggleSidebar);
    newChatBtn.addEventListener('click', createNewChat);
    clearChatBtn.addEventListener('click', clearCurrentChatMessages);

    // Settings Modal
    openSettingsBtn.addEventListener('click', () => {
        systemPromptInput.value = state.systemPrompt;
        settingsModal.classList.remove('hidden');
    });
    closeSettingsBtn.addEventListener('click', () => {
        settingsModal.classList.add('hidden');
    });
    resetPromptBtn.addEventListener('click', () => {
        systemPromptInput.value = DEFAULT_SYSTEM_PROMPT;
    });
    apiProviderSelect.addEventListener('change', updateSettingsUI);
    saveSettingsBtn.addEventListener('click', saveSettings);
}

function toggleSidebar() {
    document.getElementById('sidebar').classList.toggle('-translate-x-full');
    document.getElementById('mobile-overlay').classList.toggle('hidden');
```

```
}

function updateSettingsUI() {
    const provider = apiProviderSelect.value;
    if (provider === 'gemini') {
        apiKeyContainer.classList.remove('hidden');
        modelSelectContainer.classList.remove('hidden');
    } else {
        apiKeyContainer.classList.add('hidden');
        modelSelectContainer.classList.add('hidden');
    }

    apiProviderSelect.value = state.provider;
    apiKeyInput.value = state.apiKey;
    modelNameSelect.value = state.model;
    modeBadge.textContent = state.provider === 'gemini' ? 'Gemini API' : 'Mock Engine';
}

function saveSettings() {
    state.systemPrompt = systemPromptInput.value.trim() || DEFAULT_SYSTEM_PROMPT;
    state.provider = apiProviderSelect.value;
    state.apiKey = apiKeyInput.value.trim();
    state.model = modelNameSelect.value;

    localStorage.setItem('cfa_system_prompt', state.systemPrompt);
    localStorage.setItem('cfa_provider', state.provider);
    localStorage.setItem('cfa_api_key', state.apiKey);
    localStorage.setItem('cfa_model', state.model);

    updateSettingsUI();
    settingsModal.classList.add('hidden');
}

function createNewChat() {
    const newChat = {
        id: 'chat_' + Date.now(),
        title: 'New Conversation',
        messages: [],
```

```
        createdAt: new Date().toISOString()
    };
    state.chats.unshift(newChat);
    state.currentChatId = newChat.id;
    saveChatsToStorage();
    renderChatHistory();
    loadCurrentChat();
}

function getCurrentChat() {
    return state.chats.find(c => c.id === state.currentChatId);
}

function loadCurrentChat() {
    const chat = getCurrentChat();
    messagesContainer.innerHTML = '';

    if (!chat || chat.messages.length === 0) {
        messagesContainer.appendChild(welcomeScreen);
        welcomeScreen.classList.remove('hidden');
    } else {
        welcomeScreen.classList.add('hidden');
        chat.messages.forEach(msg => appendMessageToDOM(msg.role, msg.content));
    }
    scrollToBottom();
}

function clearCurrentChatMessages() {
    const chat = getCurrentChat();
    if (chat) {
        chat.messages = [];
        saveChatsToStorage();
        loadCurrentChat();
    }
}

function renderChatHistory() {
    historyItems.innerHTML = '';
```

```
        state.chats.forEach(chat => {
            const isActive = chat.id === state.currentChatId;
            const item = document.createElement('div');
            item.className = `group flex items-center justify-between p-2.5 rounded-xl cursor-pointer text-xs transition ${
                isActive ? 'bg-indigo-600/20 text-white font-medium border border-indigo-500/30' : 'text-gray-400 hover:bg-gray-800/60 hover:text-gray-200'
            }`;

            item.innerHTML = `
                <div class="flex items-center gap-2 truncate">
                    <i class="fa-regular fa-message text-[11px]"></i>
                    <span class="truncate max-w-[140px]">${chat.title}</span>
                </div>
                <button class="delete-chat-btn opacity-0 group-hover:opacity-100 p-1 hover:text-red-400 transition" title="Delete Chat">
                    <i class="fa-solid fa-xmark"></i>
                </button>
            `;

            item.addEventListener('click', (e) => {
                if (e.target.closest('.delete-chat-btn')) {
                    e.stopPropagation();
                    deleteChat(chat.id);
                } else {
                    state.currentChatId = chat.id;
                    renderChatHistory();
                    loadCurrentChat();
                    if (window.innerWidth < 768) toggleSidebar();
                }
            });

            historyItems.appendChild(item);
        });
    }

    function deleteChat(chatId) {
        state.chats = state.chats.filter(c => c.id !== chatId);
```

```
    if (state.chats.length === 0) {
        createNewChat();
    } else {
        if (state.currentChatId === chatId) {
            state.currentChatId = state.chats[0].id;
        }
        saveChatsToStorage();
        renderChatHistory();
        loadCurrentChat();
    }
}

function saveChatsToStorage() {
    localStorage.setItem('cfa_chats', JSON.stringify(state.chats));
}

function handleUserSendMessage(text) {
    const chat = getCurrentChat();
    if (!chat) return;

    if (chat.messages.length === 0) {
        chat.title = text.slice(0, 28) + (text.length > 28 ? '...' : '');
        renderChatHistory();
    }

    welcomeScreen.classList.add('hidden');

    const userMsg = { role: 'user', content: text, timestamp: new Date().toISOString() };
    chat.messages.push(userMsg);
    appendMessageToDOM('user', text);
    saveChatsToStorage();

    const typingId = appendTypingIndicator();
    state.isGenerating = true;
    sendBtn.disabled = true;

    generateAIResponse(text)
        .then(aiResponseText => {
```

```
                removeTypingIndicator(typingId);
                const aiMsg = { role: 'assistant', content: aiResponseText, timestamp: new
Date().toISOString() };
                chat.messages.push(aiMsg);
                appendMessageToDOM('assistant', aiResponseText);
                saveChatsToStorage();
            })
            .catch(err => {
                removeTypingIndicator(typingId);
                const errorMsg = `**Chart Fiddy Ac Notice:** Error generating response (${
{err.message}).`;
                chat.messages.push({ role: 'assistant', content: errorMsg });
                appendMessageToDOM('assistant', errorMsg);
                saveChatsToStorage();
            })
            .finally(() => {
                state.isGenerating = false;
                sendBtn.disabled = false;
            });
        }

        function appendMessageToDOM(role, content) {
            const isUser = role === 'user';
            const msgRow = document.createElement('div');
            msgRow.className = `flex gap-3 md:gap-4 max-w-3xl mx-auto ${isUser ? 'justify-end' :
'justify-start'}`;

            const avatarHTML = isUser ? `
                <div class="w-8 h-8 rounded-full bg-gray-700 flex items-center justify-center text-xs
font-semibold text-white shrink-0 order-2">
                    YOU
                </div>
            ` : `
                <div class="w-8 h-8 rounded-xl bg-gradient-to-tr from-purple-600 to-indigo-600 flex
items-center justify-center text-white text-xs font-bold shrink-0 shadow-md">
                    AC
                </div>
            `;
```

```
        const parsedContent = isUser ? escapeHTML(content) : marked.parse(content);

        const bubbleHTML = `
            <div class="flex-1 max-w-[85%] ${isUser ? 'order-1' : 'order-2'}">
                <div class="text-[11px] font-semibold text-gray-500 mb-1 ${isUser ? 'text-right' :
'text-left'}">
                    ${isUser ? 'You' : 'Chart Fiddy Ac'}
                </div>
                <div class="p-4 rounded-2xl text-sm ${
                    isUser
                        ? 'bg-indigo-600 text-white rounded-tr-none shadow-md'
                        : 'bg-darkcard border border-gray-800 text-gray-200 rounded-tl-none prose
shadow-sm'
                }">
                    ${parsedContent}
                </div>
            </div>
        `;

        msgRow.innerHTML = isUser ? (bubbleHTML + avatarHTML) : (avatarHTML +
bubbleHTML);
        messagesContainer.appendChild(msgRow);

        if (!isUser) {
            msgRow.querySelectorAll('pre').forEach(pre => {
                const copyBtn = document.createElement('button');
                copyBtn.className = 'absolute top-2 right-2 text-xs bg-gray-800 hover:bg-
gray-700 text-gray-300 py-1 px-2 rounded transition';
                copyBtn.innerHTML = '<i class="fa-regular fa-copy mr-1"></i> Copy';
                copyBtn.addEventListener('click', () => {
                    const codeText = pre.querySelector('code').innerText;
                    navigator.clipboard.writeText(codeText);
                    copyBtn.innerHTML = '<i class="fa-solid fa-check mr-1 text-emerald-400"></i>
Copied!';
                    setTimeout(() => copyBtn.innerHTML = '<i class="fa-regular fa-copy mr-1"></i>
Copy', 2000);
                });
```

```
            pre.appendChild(copyBtn);
        });
    }

    scrollToBottom();
}

function appendTypingIndicator() {
    const id = 'typing_' + Date.now();
    const msgRow = document.createElement('div');
    msgRow.id = id;
    msgRow.className = 'flex gap-3 md:gap-4 max-w-3xl mx-auto justify-start';

    msgRow.innerHTML = `
        <div class="w-8 h-8 rounded-xl bg-gradient-to-tr from-purple-600 to-indigo-600 flex items-center justify-center text-white text-xs font-bold shrink-0 shadow-md">
            AC
        </div>
        <div class="p-4 rounded-2xl bg-darkcard border border-gray-800 text-gray-400 rounded-tl-none flex items-center gap-1.5">
            <span class="w-2 h-2 rounded-full bg-indigo-500 typing-dot"></span>
            <span class="w-2 h-2 rounded-full bg-indigo-500 typing-dot"></span>
            <span class="w-2 h-2 rounded-full bg-indigo-500 typing-dot"></span>
            <span class="text-xs text-gray-500 ml-2">Chart Fiddy Ac is thinking...</span>
        </div>
    `;

    messagesContainer.appendChild(msgRow);
    scrollToBottom();
    return id;
}

function removeTypingIndicator(id) {
    const el = document.getElementById(id);
    if (el) el.remove();
}

function scrollToBottom() {
```

```
        messagesContainer.scrollTop = messagesContainer.scrollHeight;
    }

    function escapeHTML(str) {
        return str.replace(/[&<>'"]/g,
            tag => ({ '&': '&amp;', '<': '&lt;', '>': '&gt;', "'": '&#39;', '"': '&quot;' }[tag] || tag)
        );
    }

    async function generateAIResponse(userPrompt) {
        if (state.provider === 'gemini' && state.apiKey) {
            return await callGeminiAPI(userPrompt);
        } else {
            return await generateMockResponse(userPrompt);
        }
    }

    async function callGeminiAPI(prompt) {
        const url = `https://generativelanguage.googleapis.com/v1beta/models/${state.model}:generateContent?key=${state.apiKey}`;

        const payload = {
            contents: [
                {
                    role: 'user',
                    parts: [{ text: `${state.systemPrompt}\n\nUser Message:\n${prompt}` }]
                }
            ]
        };

        const response = await fetch(url, {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify(payload)
        });

        if (!response.ok) {
            const errData = await response.json();
```

```
            throw new Error(errData.error?.message || 'API request failed');
        }

        const data = await response.json();
        return data.candidates?.[0]?.content?.parts?.[0]?.text || "I couldn't process that response.";
    }

    function generateMockResponse(prompt) {
        return new Promise((resolve) => {
            setTimeout(() => {
                const lower = prompt.toLowerCase();
                let response = "";

                // Creator Question Handling
                if (lower.includes('who created') || lower.includes('who made') || lower.includes('who built') || lower.includes('creator')) {
                    response = `### 👑 Creator Identity

I was created, designed, and developed by **Fiddy**!

Fiddy designed me to be a warm, supportive, and highly versatile AI companion—combining structured academic support, creative arts guidance, and digital design brainstorming, while always honoring and protecting original creative work. How can I assist you today?`;
                } else if (lower.includes('chemistry') || lower.includes('science') || lower.includes('math')) {
                    response = `### 🧪 Academic Breakdown: ${prompt.slice(0, 30)}...

I am glad to help you tackle this! Let's break down the concepts clearly:

#### Key Principles:
1. **Fundamental Rule:** Focus first on understanding core definitions before attempting complex calculations.
2. **Methodology:**
   - Define all known variables.
   - Set up your equation carefully.
   - Verify units at every step.
```

Tip from Chart Fiddy Ac: Keep practicing examples and test yourself without looking at the solutions!`;
                } else {
                    response = `### 🌟 Hello! Chart Fiddy Ac Here

Thank you for reaching out! I am right here with you, created by **Fiddy** to assist across academics, creative writing, digital design, or brainstorming new ideas.

How can we collaborate on **${prompt}**? Let's dive in!`;
                }

                resolve(response);
            }, 1000);
        });
    }

    window.addEventListener('DOMContentLoaded', initApp);
</script>
</body>
</html>